

PAR Laboratory Assignment
Lab 3: Analysis of parallel strategies —
The computation of the Mandelbrot set

Mario Acosta, Eduard Ayguadé, Rosa M. Badia, Jesús Labarta, Pau Ferrer,
Daniel Jiménez-González, Pablo Vizcaino, and Gladys Utrera

Spring 2025–2026



UNIVERSITAT POLITÈCNICA DE CATALUNYA
BARCELONATECH

Departament d'Arquitectura de Computadors

Contents

1	A short introduction to the Mandelbrot set	5
2	Experimental setup	7
2.1	Laboratory files	7
2.2	Mandelbrot parameters	7
2.3	Compilation	8
2.4	Execution	8
2.5	Analysis	8
3	Iterative task decomposition analysis	9
3.1	Sequential execution	9
3.2	Analysis of the different parallel strategies	9
4	Recursive task decomposition analysis	11
4.1	Sequential execution	11
4.2	Analysis of the different parallel strategies	12
A	Laboratory deliverable information	13
A.1	Individual results for each parallel strategy	13
A.2	Final comparison between all the parallel strategies	13
A.3	Evaluation of English as a third language	14
A.4	Submission of the deliverable and source codes to Atenea	14

Note. Chapters 3 and 4 of this document correspond each to a laboratory session (2 hours).

Important!

Your objective is to analyze and/or modify the provided code of Mandelbrot, for each parallel strategy we ask for, to determine the dependencies between the tasks and analyse the potential concurrency problems due to data sharing, synchronisation, and possible load unbalance. Keep reading all the document, having in mind the instructions of Annex A to know the results and analysis we ask you for the deliverable report. Annex A describes the laboratory document that you will need to elaborate and upload to Atenea together with all your C source codes.

Chapter 1

A short introduction to the Mandelbrot set

In this laboratory assignment you will explore different parallel strategies based on *iterative* and *recursive* task decompositions. You will compute the “Mandelbrot set”, that is, a particular set of points in the complex domain, whose boundary generates a distinctive and easily recognisable two-dimensional fractal shape (see Fig. 1.1).

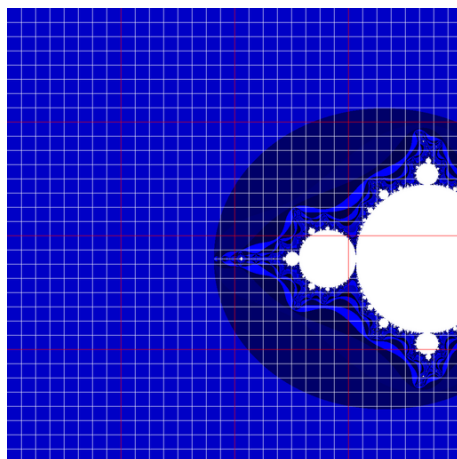


Figure 1.1: Numerical grid containing the fractal shape of the Mandelbrot set.

For each point c in a delimited two-dimensional space, the complex quadratic polynomial recurrence $z_{n+1} = z_n^2 + c$ is iteratively applied n times to determine if it belongs or not to the Mandelbrot set. The point c is part of the Mandelbrot set if, when starting with $z_0 = 0$ and applying the polynomial recurrence repeatedly, the absolute value of z_n never exceeds a certain threshold (e.g. 2) however large n gets.

A plot of the Mandelbrot set is created by colouring each point c in the complex plane with the number of steps for which $|z_{\max}| \geq 2$ (or simply $|z_{\max}|^2 \geq 4$ to avoid the computation of the square root in the modulus of a complex number). In order to make the problem doable, the maximum number of steps $\max$ is limited. Therefore the point c is said to belong to the Mandelbrot set when the maximum number of steps is reached.

If you want to know more about the Mandelbrot set, we recommend you to take a look at the following Wikipedia page:

https://en.wikipedia.org/wiki/Mandelbrot_set

You will find different ways to render the Mandelbrot set. In particular, we will be using the

idea of Mariani’s algorithm¹ to render the Mandelbrot set. This method checks the borders of each rectangle (i.e. tile) thereby avoiding the expensive computation associated with a full tile: if all the points along the borders share the same value, then all the interior points do also share the same convergence values. Figure 1.1 shows a tiled version of the Mandelbrot picture: tiles that fall within an area of the same colour can be computed by only copying the value obtained from its borders.

¹Dewdney, A. K. (1989). “Computer Recreations, February 1989; A tour of the Mandelbrot set aboard the Mandelbus”. *Scientific American*. p. 111. JSTOR 24987149

Chapter 2

Experimental setup

2.1 Laboratory files

This laboratory assignment will be done in two sessions of two hours each. All the files necessary to carry out this laboratory are available in a compressed file at `/scratch/nas/1/par0/sessions/lab3.tar.gz` located in `boada.ac.upc.edu`.

Uncompress the aforementioned file in your home directory with these command lines:

```
cd
tar -zxvf ~par0/sessions/lab3.tar.gz
```

You will find the following files in the `lab3` directory:

- Makefile.
- Source codes:
 - mandel-seq-iter.c:** Iterative sequential code without instrumentation.
 - mandel-seq-rec.c:** Recursive sequential code without instrumentation.
 - mandel-seq-iter-tar.c:** Iterative sequential code *with the iterative, original* Tareador instrumentation.
 - mandel-seq-rec-tar.c:** Recursive sequential code *with essential* Tareador instrumentation.
- Scripts:
 - submit-seq.sh:** Script to execute the sequential code and useful to obtain the expected Mandelbrot outputs.
 - run-tareador.sh:** Script to execute the Tareador instrumented sequential code.

2.2 Mandelbrot parameters

The arguments to be pass to the executables can be seen by running the command `mandel -help`:

```
./mandel-seq-iter -help
Usage: ./mandel-seq-iter [-o -h -d -i maxiter -c x0 y0 -s size]
  -o to write computed image (mandel_image.jpg) and histogram \
      (mandel_histogram.out if -h indicated) to disk (default no file generated)
  -h to produce histogram of values in computed image (default no histogram)
  -d to display computed image (default no display)
  -i to specify maximum number of iterations at each point (default 100)
  -c to specify the center x0+iy0 of the square to compute
  -s to specify the size of the square to compute (default 2, i.e. size 4 by 4)
```

Warning!

The `-o` option makes the program to write the image and histogram to disk always using with the *exact same* name. Therefore, rename them if you want to keep them and later compare them with your future parallel outputs. Both of these files are stored in binary format so that you will need to use `cmp` or `diff` commands to compare them.

2.3 Compilation

Look at the `Makefile` to understand how to compile each executable.

2.4 Execution

We have provided you with the Tareador scripts to help you analyse your parallel strategies.

2.5 Analysis

We suggest you to review the Lab 1 session 2, and annexes to remember the functionality of Tareador and how to use it to generate and analyse your parallel strategies. You need to remember how to disable the analysis of a given variable in Tareador, as it will affect the parallelism of your application.

Chapter 3

Iterative task decomposition analysis

The iterative “tile” pseudocode of the Mandelbrot set computation is shown in Code 1.

```
for (int y=0; y<numROW ; y+=TILE) {
  for (int x=0; x<numCOL; x+=TILE) {
    equal = ...           // Check if vertical TILE borders have the same colour
    equal = equal & ...   // Check if horizontal TILE borders have the same colour
    if (equal) {          // Fill the full TILE with the same colour
      fill_tile_code      // (if -d, display, if -h, update histogram)
    } else {              // Compute the full TILE in the Mandelbrot set
      compute_tile_code   //(if -d, display, if -h, update histogram)
    }
  }
}
```

Code 1: Iterative tile version of the sequential code computing the Mandelbrot set.

3.1 Sequential execution

Run the following commands to figure out what it produces:

- Run the Mandelbrot set to only measure its execution time:
`sbatch submit-seq.sh ./mandel-seq-iter -i 10000`
- Run it to generate the histogram (`mandel_histogram.out`) and image (`mandel_image.jpg`) outputs:
`sbatch submit-seq.sh ./mandel-seq-iter -h -o -i 10000`
- Run it interactively with the display option:
`./mandel-seq-iter -d -i 10000`
- How to run the Tareador analysis of the program is shown in the next section.

3.2 Analysis of the different parallel strategies

The code `mandel-seq-iter-tar.c` has already been instrumented with Tareador to implement a straightforward parallel strategy: each iteration of loop along `x` is a task.

We ask you to analyse the following three parallel strategies:

Original tile parallel strategy: each iteration of the loop along x is a task. You need to perform three analyses:

1. Run it with no arguments:

```
./run-tareador.sh mandel-seq-iter-tar
```

Use `tareador_disable_object(&var_name)` to disable the dependencies on a given variable due to data sharing in order to explore the parallelism among tasks whenever possible.

2. Run it with only the `-d` option to display the Mandelbrot set picture.

```
./run-tareador.sh mandel-seq-iter-tar -d
```

Note that you have to close (or press any key to close) the window displaying the Mandelbrot set to allow Tareador to finish its work. Use `tareador_disable_object(&var_name)` to disable the dependencies on a given variable due to data sharing in order to explore the parallelism among tasks whenever possible.

3. Run it with only the `-h` option to keep the histogram of colours.

```
./run-tareador.sh mandel-seq-iter-tar -h
```

Use `tareador_disable_object(&var_name)` to disable the dependencies on a given variable due to data sharing in order to explore the parallelism among tasks whenever possible.

The analysis carried out in this section by running with either `-d` or `-h` options is particularly useful for the remaining parallel strategies. Therefore, we do not ask you (and you do not need to) run the remaining strategies with `-d` and `-h`. However, you should take into account the conclusions obtained above.

Fine grain parallel strategy: Remove the Tareador task definition corresponding to the previous original tile strategy and add the following new parallel strategy with 5 defined tasks (the comments within the code indicate each part):

1. Check vertical borders is a task.
2. Check horizontal borders is a task.
3. Entire if-else conditional (after both checks) is another task and this includes two other task definitions:
 - (a) each `py` iteration of fill all with the same value is a task, and
 - (b) each `py` iteration of computation of a tile is a task.

You only need to perform the analysis running this strategy with no arguments. Use `tareador_disable_object(&var_name)` to disable the dependencies on a given variable due to data sharing in order to explore the parallelism among tasks whenever possible.

Column of tiles parallel strategy: Assuming that the first strategy (original tile strategy) is your baseline source code, remove the task from this strategy and interchange the x and the y loops. Next, instrument the modified code so that each x iteration is a task.

Again, you only need to perform the analysis running this strategy with no arguments. Use `tareador_disable_object(&var_name)` to disable the dependencies on a given variable due to data sharing in order to explore the parallelism among tasks whenever possible.

Chapter 4

Recursive task decomposition analysis

The recursive pseudocode of the Mandelbrot set computation is shown in Code 2.

```
recursive_call(matrix, tile) {
    equal = ...           // Check if vertical borders have the same colour
    equal = equal & ...   // Check if horizontal borders have the same colour
    if (equal) {          // Fill the full matrix tile with the same colour
        fill_tile_code    // (if -d, display, if -h, update histogram)
    } else {
        if (matrix_tile_size < TILE) { // Computation of matrix tile (Mandelbrot set)
            compute_tile_code          // (if -d, display, if -h, update histogram)
        } else {
            recursive_call(left-top subtile of matrix tile)
            recursive_call(left-bottom subtile of matrix tile)
            recursive_call(right-top subtile of matrix tile)
            recursive_call(right-bottom subtile of matrix tile)
        }
    }
}
```

Code 2: Tile version of the recursive sequential code computing the Mandelbrot set.

4.1 Sequential execution

Run the following commands to figure what it produces:

- Run the Mandelbrot set to only measure its execution time:

```
sbatch submit-seq.sh ./mandel-seq-rec -i 10000
```

- Run it to generate histogram (mandel_histogram.out) and image (mandel_image.jpg) outputs:

```
sbatch submit-seq.sh ./mandel-seq-rec -h -o -i 10000
```

- Run it interactively with the display option:

```
./mandel-seq-rec -d -i 10000
```

4.2 Analysis of the different parallel strategies

The code `mandel-seq-rec-tar.c` has only been instrumented with the necessary `tareador_ON` and `tareador_OFF` calls. We ask you to instrument it with Tareador in order to evaluate and analyse the leaf and tree parallel strategies:

Leaf recursive parallel strategy. Remember that only base cases of the recursive program are tasks.

Use `tareador_disable_object(&var_name)` to disable the dependencies on a given variable due to data sharing in order to explore the parallelism among tasks whenever possible.

Run Tareador:

Warning!

Do not run Tareador until you have added all the tasks. Note also that the Tareador execution may take a while.

```
./run-tareador.sh mandel-seq-rec-tar
```

Tree recursive parallel strategy. In this case we ask you to create tasks following a recursive task decomposition, but also exploit the parallelism between both the vertical and horizontal border check tasks.

Use `tareador_disable_object(&var_name)` to disable the dependencies on a given variable due to data sharing in order to explore the parallelism among tasks whenever possible. Run Tareador:

```
./run-tareador.sh mandel-seq-rec-tar
```

Appendix A

Laboratory deliverable information

A.1 Individual results for each parallel strategy

For each strategy that you analyse in this laboratory, Table A.1 explains what should be included in the report.

Table A.1: Analysis to be included in the PDF deliverable report for each parallel strategy (and execution argument in the case of the “tile strategy”).

Section	Description
Task decomposition	Name of the parallel strategy
Arguments	Arguments used during the execution.
Code	Add the source code to the compressed file. Indicate the name of the source code in the PDF. Please refrain from including the entire code in the deliverable document, except for fragments of codes that you consider necessary in order to explain your work.
Tareador TDG (with dependencies)	Include the TDG image for the strategy (run arguments) with all the dependencies.
Dependency analysis	Explain the dependencies found indicating the name of the variable(s). Reason whether they force the order of the tasks or if you may be able to use data sharing synchronisations instead.
Tareador TDG (without “data sharing” dependencies)	Include a new TDG image obtained when the dependencies that can be removed using data sharing synchronisations are deactivated with Tareador (<code>tareador_disable_object(&var_name)</code>).
T_∞ analysis	Assuming that you have disabled only data sharing dependencies, perform the corresponding simulations to describe whether there may be load unbalance. Compute also T_1 and T_∞ by clicking on View Simulation in the Tareador window with 1 and 128 threads, respectively.

A.2 Final comparison between all the parallel strategies

Table A.2 should be included after the description of all the parallel strategies. The row “Run Arguments” in the table refers to the optional arguments to be passed to `run-tareador.sh strategy_code arguments`. Based on your parallel strategy results, reason which will be the best parallel strategy for iterative and recursive implementations.

Table A.2: Summary of the parallel performance for each parallel strategy to be included in your PDF deliverable report.

Task decomposition	Strategy	Run arguments	T_1	T_∞	Parallelism	Load unbalance: yes/no, why?
Iterative	Original					
	Original	-d				
	Original	-h				
	Fine grain					
	Column					
Recursive	Leaf					
	Tree					
Best task decomposition	Strategy	Reason why				

A.3 Evaluation of English as a third language

As you should know, this course contributes to the transversal competence “Tercera llengua”. Please deliver your material in English if you want this competence to be evaluated and also refer to the “Rubrics for the third language competence evaluation” document to know which rubrics will be utilised.

A.4 Submission of the deliverable and source codes to Atenea

An entry for the submission will be created in Atenea and the proper submission deadlines will be set. Only a PDF format for this document will be accepted for submission. Additionally, you are required to submit complete source codes with the Tareador instrumentation that you have developed. Please refrain from including the entire source code in the deliverable document, except for fragments of codes that you consider necessary to explain your work.

Important!

You must upload two separate files to Atenea:

1. The deliverable document in PDF format.
2. A compressed file (zip or tar.gz) containing all the requested source codes.